

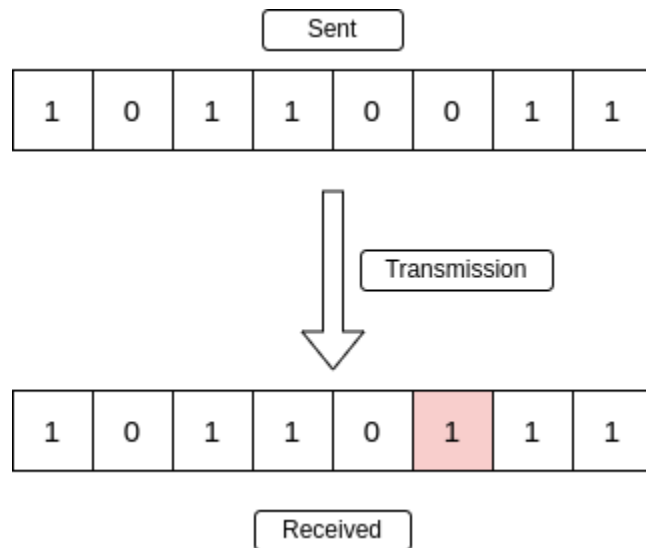
## Error Detection and Correction

In computer networks and communication systems, **error detection and correction** are essential to ensure that the transmitted data is reliable and accurate. Errors can occur during data transmission due to noise, signal interference, or physical medium issues. **Error detection** allows the receiver to detect errors in the received data, while **error correction** enables the system to recover from those errors.

### Types of Errors

#### Single-Bit Error

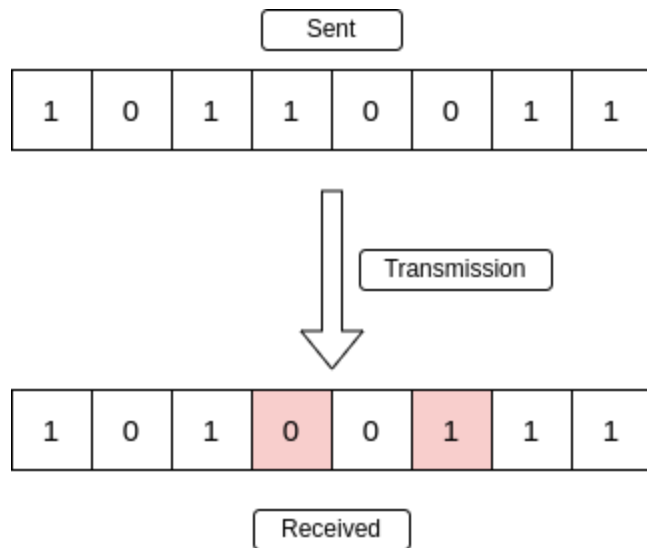
A single-bit error refers to a type of data transmission error that occurs when one bit (i.e., a single binary digit) of a transmitted data unit is altered during transmission, resulting in an incorrect or corrupted data unit.



*Single-Bit Error*

#### Multiple-Bit Error

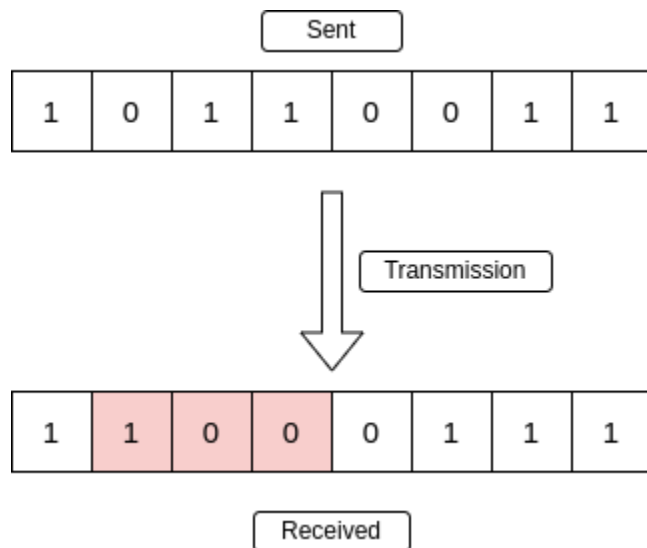
A multiple-bit error is an error type that arises when more than one bit in a data transmission is affected. Although multiple-bit errors are relatively rare when compared to single-bit errors, they can still occur, particularly in high-noise or high-interference digital environments.



*Multiple-Bit Error*

### **Burst Error**

When several consecutive bits are flipped mistakenly in digital transmission, it creates a burst error. This error causes a sequence of consecutive incorrect values.



*Burst Error*

### **Random error:**

- Random bit flips occur due to noise or interference in the transmission medium. It can affect one or more bits in a non-predictable manner.

## Redundancy

**Redundancy** is the key principle behind most error detection and correction techniques. It involves adding extra bits to the transmitted data so that the receiver can detect and possibly correct errors. These extra bits, known as **redundant bits**, provide additional information that helps verify the integrity of the received data.

For example, in the **parity bit** method, an additional bit is added to make the total number of 1s either even or odd, allowing for error detection.

## Error Detection Methods

To detect errors, a common technique is to introduce redundancy bits that provide additional information. Various techniques for error detection include:

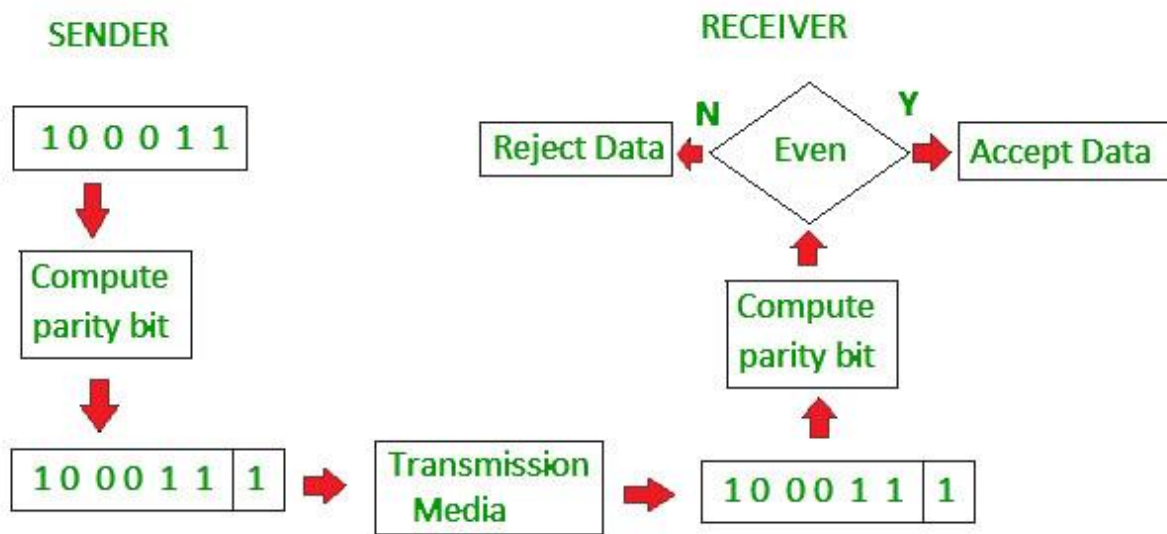
- Simple Parity Check
- Two-Dimensional Parity Check
- Checksum
- Cyclic Redundancy Check (CRC)
- **Vertical Redundancy Check (VRC):**
- **Longitudinal Redundancy Check (LRC):**

## Simple Parity Check

Simple-bit parity is a simple error detection method that involves adding an extra bit to a data transmission. It works as:

- 1 is added to the block if it contains an odd number of 1's, and
- 0 is added if it contains an even number of 1's

This scheme makes the total number of 1's even, that is why it is called [even parity](#) checking.



### Advantages of Simple Parity Check

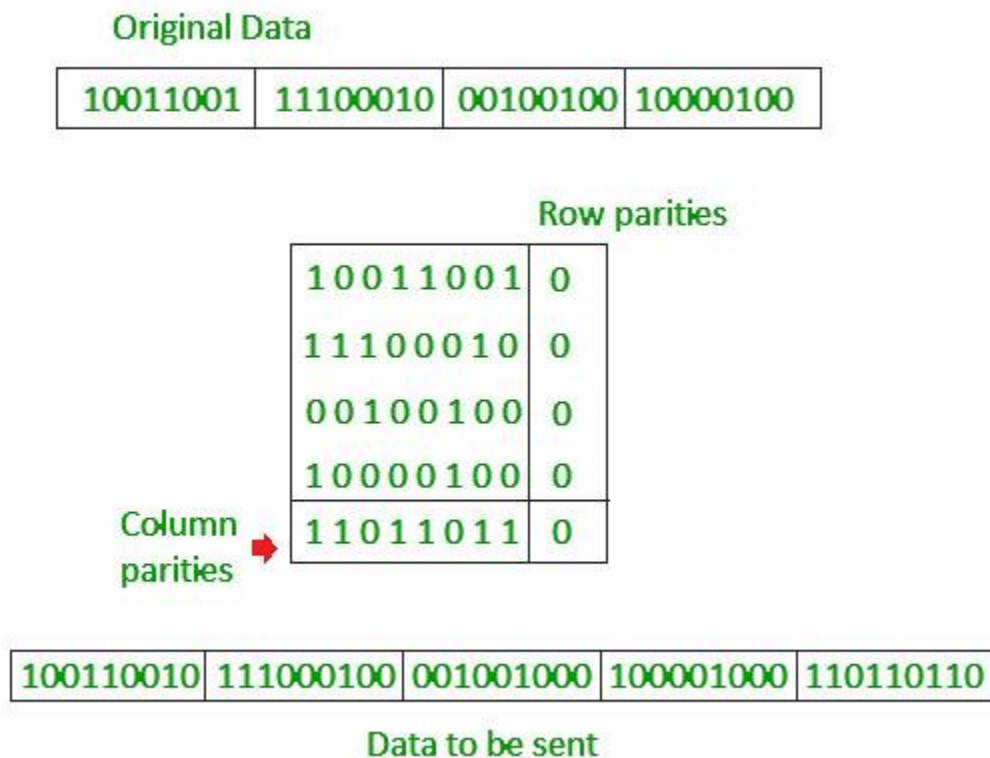
- Simple parity check can detect all single bit error.
- Simple parity check can detect an odd number of errors.
- **Implementation:** Simple Parity Check is easy to implement in both hardware and software.
- **Minimal Extra Data:** Only one additional bit (the parity bit) is added per data unit (e.g., per byte).
- **Fast Error Detection:** The process of calculating and checking the parity bit is quick, which allows for rapid error detection without significant delay in data processing or communication.
- **Single-Bit Error Detection:** It can effectively detect single-bit errors within a data unit, providing a basic level of error detection for relatively low-error environments.

### Disadvantages of Simple Parity Check

- Single Parity check is not able to detect even no. of bit error.
- **For example,** the Data to be transmitted is **101010**. Codeword transmitted to the receiver is 1010101 (we have used even parity).  
Let's assume that during transmission, two of the bits of code word flipped to 1111101. On receiving the code word, the receiver finds the no. of ones to be even and hence **no error**, which is a wrong assumption.

## Two-Dimensional Parity Check

**Two-dimensional Parity check** bits are calculated for each row, which is equivalent to a simple parity check bit. Parity check bits are also calculated for all columns, then both are sent along with the data. At the receiving end, these are compared with the parity bits calculated on the received data.



### Advantages of Two-Dimensional Parity Check

- Two-Dimensional Parity Check can detect and correct all single bit error.
- Two-Dimensional Parity Check can detect two or three bit error that occur anywhere in the matrix.

### Disadvantages of Two-Dimensional Parity Check

- Two-Dimensional Parity Check can not correct two or three bit error. It can only detect two or three bit error.
- If we have an error in the parity bit then this scheme will not work.

### Checksum

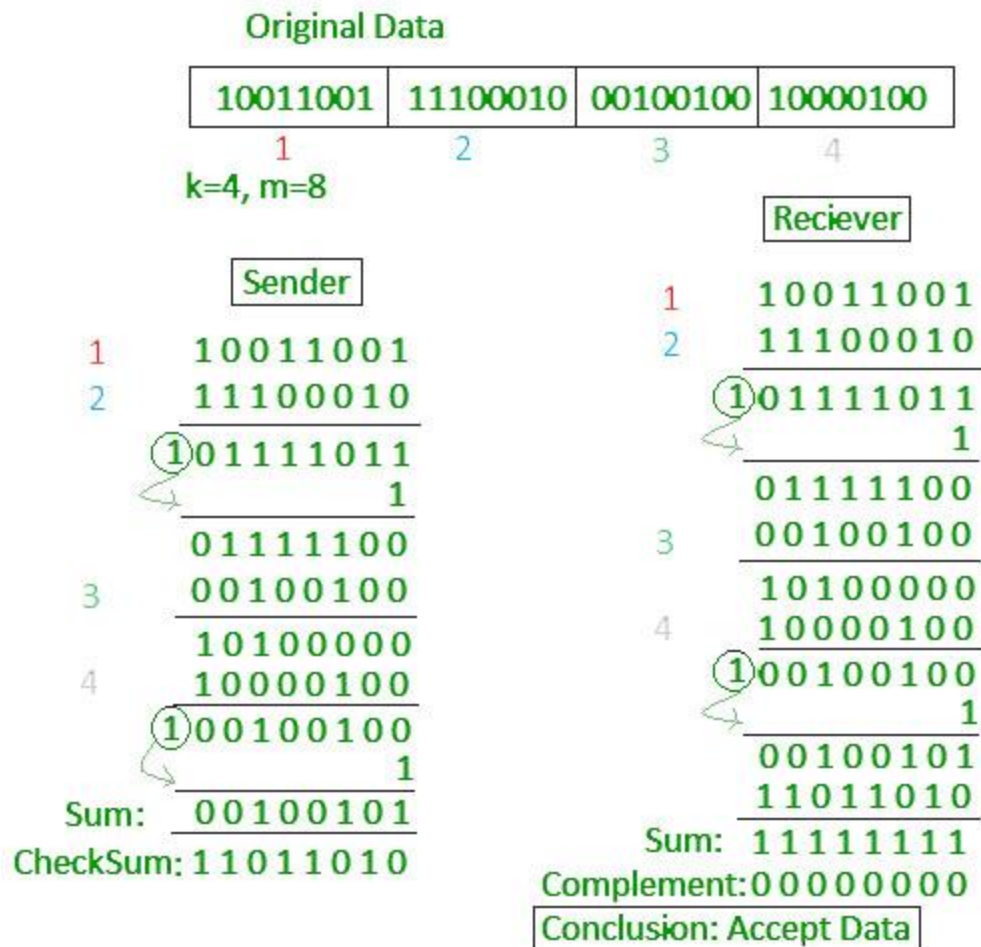
Checksum error detection is a method used to identify errors in transmitted data. The process involves dividing the data into equally sized segments and using a [1's complement](#) to calculate the sum of these segments. The calculated sum is then sent along with the data to the receiver. At the receiver's end, the same process is repeated and if all zeroes are obtained in the sum, it means that the data is correct.

#### **Checksum – Operation at Sender's Side**

- Firstly, the data is divided into k segments each of m bits.
- On the sender's end, the segments are added using 1's complement arithmetic to get the sum. The sum is complemented to get the checksum.
- The checksum segment is sent along with the data segments.

#### **Checksum – Operation at Receiver's Side**

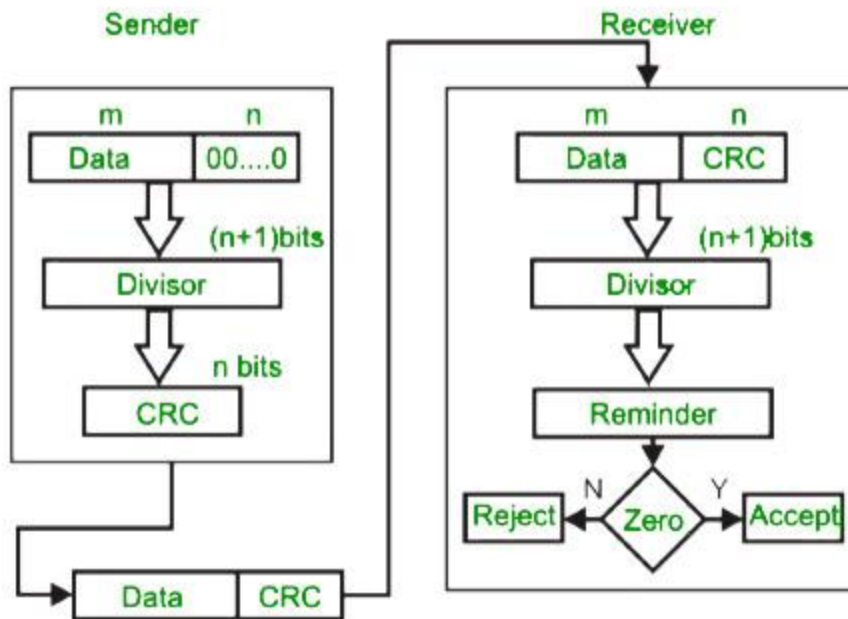
- At the receiver's end, all received segments are added using 1's complement arithmetic to get the sum. The sum is complemented.
- If the result is zero, the received data is accepted; otherwise discarded.



Read more about [Checksum](#)

### Cyclic Redundancy Check (CRC)

- Unlike the checksum scheme, which is based on addition, CRC is based on [binary division](#).
- In CRC, a sequence of redundant bits, called cyclic redundancy check bits, are appended to the end of the data unit so that the resulting data unit becomes exactly divisible by a second, predetermined binary number.
- At the destination, the incoming data unit is divided by the same number. If at this step there is no remainder, the data unit is assumed to be correct and is therefore accepted.
- A remainder indicates that the data unit has been damaged in transit and therefore must be rejected.



### CRC Working

We have given dataword of length  $n$  and divisor of length  $k$ .

**Step 1:** Append  $(k-1)$  zero's to the original message

**Step 2:** Perform modulo 2 division

**Step 3:** Remainder of division = CRC

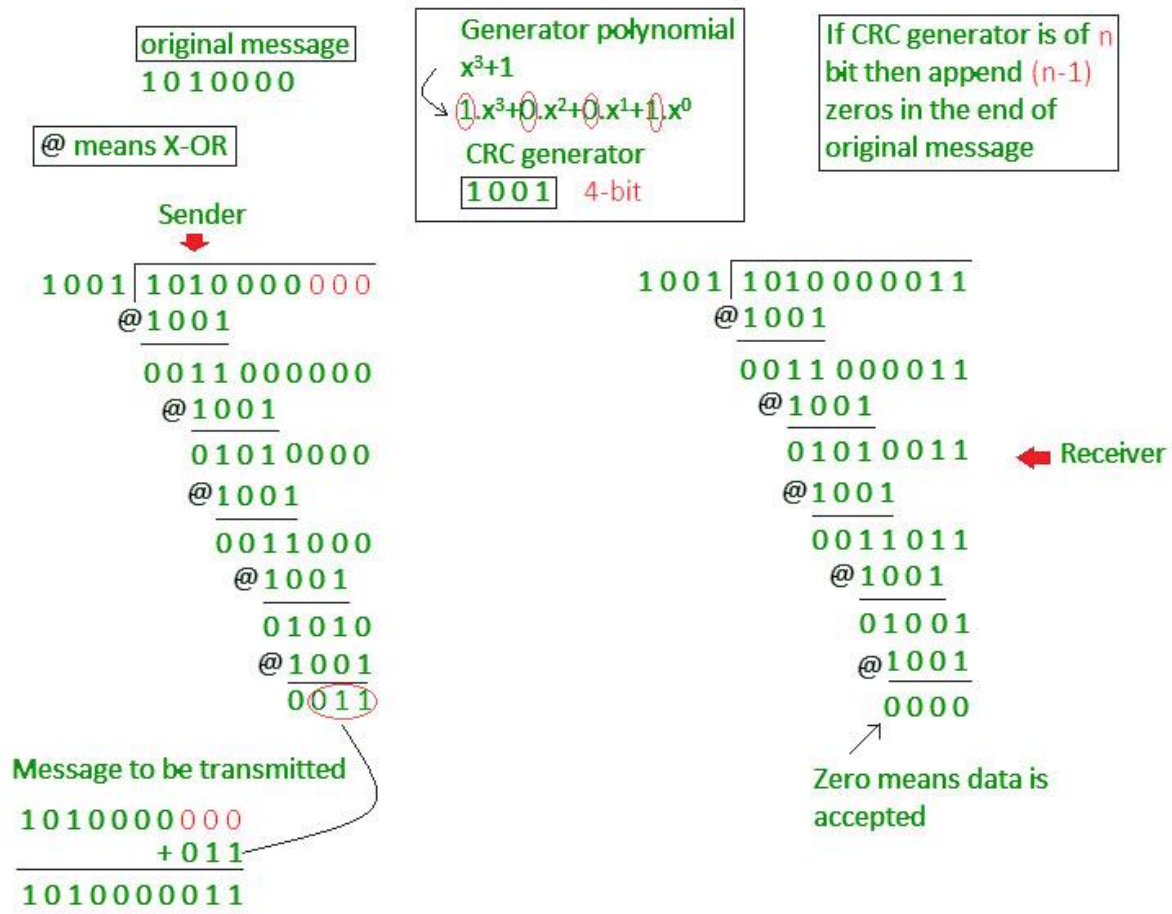
**Step 4:** Code word = Data with append  $k-1$  zero's + CRC

Note:

- CRC must be  $k-1$  bits
- Length of Code word =  $n+k-1$  bits

Example: Let's data to be send is 1010000 and divisor in the form of polynomial is  $x^3+1$ . CRC method discussed below.





Read in detail about [Cyclic Redundancy Check\(CRC\)](#)

Previous year GATE questions based on error detection: [GATE CS 2009 Question 48](#) [GATE CS 2007 Question 68](#).

### Advantages of Error Detection

- **Increased Data Reliability:** Error detection ensures that the data transmitted over the network is reliable, accurate, and free from errors. This ensures that the recipient receives the same data that was transmitted by the sender.
- **Improved Network Performance:** Error detection mechanisms can help to identify and isolate network issues that are causing errors. This can help to improve the overall performance of the network and reduce downtime.
- **Enhanced Data Security:** Error detection can also help to ensure that the data transmitted over the network is secure and has not been tampered with.

### Disadvantages of Error Detection

- **Overhead:** Error detection requires additional resources and processing power, which can lead to increased overhead on the network. This can result in slower network performance and increased latency.
- **False Positives:** Error detection mechanisms can sometimes generate false positives, which can result in unnecessary retransmission of data. This can further increase the overhead on the network.
- **Limited Error Correction:** Error detection can only identify errors but cannot correct them. This means that the recipient must rely on the sender to retransmit the data, which can lead to further delays and increased network overhead.

### **Vertical Redundancy Check (VRC):**

**VRC** is a type of **parity check** where each column of bits is checked for an even or odd number of 1s. A parity bit is appended to each column of data, ensuring the overall number of 1s is even or odd.

- **Example:**

makefile

Copy

Data: 1 0 1 0 1

1 1 0 1 0

0 0 1 1 1

Parity: 1 1 0 0 0

**Limitation:** VRC only detects errors in the vertical direction, but it is effective for simple error detection in each data block.

### **Longitudinal Redundancy Check (LRC):**

**LRC** is a more robust version of VRC, where redundancy checks are applied across both rows and columns. It ensures data integrity by checking the parity in both directions.

- **Example:** Similar to VRC, but with an additional step of checking the horizontal parity and ensuring the overall integrity of the data.

**Limitation:** Like VRC, it can only detect errors but cannot correct them.

## Detection vs. Correction

- **Error Detection:** The process of identifying that an error has occurred during transmission. Techniques like **parity bits**, **checksum**, and **CRC** are examples of error detection methods.
- **Error Correction:** The process of fixing or recovering from errors without needing a retransmission. Error correction requires more sophisticated algorithms like **Hamming codes** or **Reed-Solomon codes**.

## Forward Error Correction (FEC) vs Retransmission

- **Forward Error Correction (FEC):** FEC allows the receiver to detect and correct errors without needing a retransmission. It uses redundant bits to correct errors at the receiver side. FEC is used in environments where retransmission is not feasible (e.g., satellite communications, streaming media).

## Example: Hamming Codes

- Hamming codes are a form of FEC that uses additional bits to detect and correct errors. By adding redundant bits to the data, the receiver can detect which bit is erroneous and correct it.
- **Hamming Distance:** The number of differing bits between two codewords. The Hamming distance determines how many errors can be detected and corrected. For example, a Hamming code with a distance of 3 can detect up to 2 errors and correct 1 error.
- **Retransmission:** In contrast to FEC, **retransmission** involves requesting the sender to resend the data if an error is detected. Protocols like **TCP** use retransmission for error correction, which can result in higher latency but is simpler to implement than FEC.

---

## Hamming Codes

**Hamming codes** are a set of error-correcting codes that can detect and correct single-bit errors. It uses redundant bits that are placed at specific positions in the data stream to allow error detection and correction.

- **How Hamming Code Works:**
  - Add redundant parity bits at positions that are powers of 2 (1st, 2nd, 4th, 8th, etc.).

- These parity bits cover specific bits in the data.
  - If the number of errors is odd, the parity bits will indicate which bit is in error, allowing the receiver to correct the error.
- **Hamming Distance:** The minimum number of bit positions in which two valid codewords differ. A code with a Hamming distance of 3 can detect 2-bit errors and correct 1-bit errors.